

Port Scanning :- process of sending packets to many ports and analyzing the responses. Its used to identify open ports, services running on a target system.

Types of Port Scans

1) TCP SYN scan :-

Suppose the target is 192.168.1.10

Nmap creates a custom packet and sends it to the target (port 80 as an example)

If in the target system any application is listening on port 80 the target responds with SYN-ACK as we send the SYN custom packet first

As soon as nmap receives the SYN-ACK packet it marks port 80 = OPEN.

Instead of sending the final ACK the scanner sends [RST → terminating connection flag]
flag

The target can also use RST flag if no application is listening on requested port & send RST, ACK, as nmap receives it marks port 80 = CLOSED

2) TCP connect scan:- Here the connection is fully established and then terminated.

Nmap asks the OS to ~~send~~ create & send packets to the target. Handshake is done and connection is established

OS responds $\rightarrow$ connection succeeded & nmap marks port 80 = OPEN

Here we are not creating custom packets with nmap so OS handles the process which is convenient when being a normal user & not root.

As soon as nmap learns result the connection is terminated. It asks OS $\rightarrow$ "close this connection", then OS sends FIN or RST.

[Note:- It's the creation ~~when~~ which marks the difference. On both SYN & connect scan OS handles the delivery but in connect OS creates the packets as well, In SYN it's the scanner that creates packets

Other types of port scan:-

UDP scan, FIN scan, NULL scan, etc.

DNS

Domain Name System → Its job is to translate human readable names like google.com, youtube.com into IP addresses

Flow



U type google.com, browser checks in cache
if not asks the OS. OS checks its cache
if not found it checks the hosts file before
if not found sends a query to a ~~configured~~ DNS

Server which is configured.



This configured DNS server is called recursive server or recursive resolver, it does all the work to find the IP and returns the final answer to you



DNS server checks its cache if not available
It asks the root servers. The root servers don't know the IP but they know the top-level domain servers.

top-level domain = (.com) (.org) (.net) (.in)

The root server says ask the .com TLD server



Our recursive ^{DNS} server now asks the .com TLD server. The .com server replies "~~Ask Google~~" also doesn't know the ip but it knows the authoritative server that knows the ip and other info for the domain.



The .com TLD server lists out the authoritative servers, ex:-

ns1.google.com
ns2.google.com



Now our recursive DNS server asks the authoritative server and finally gets the reply with the ip along with TTL (Time to Live)

TTL → basically how long you are allowed to keep this DNS answer in your cache

If TTL = 300 seconds, for the next 5 minutes our resolver can answer google's ip without asking the authoritative google's server.



Finally our recursive resolver sends ip to our device. OS receives and gives it to browser



DONE

DNS records

DNS records are simply information associated with a domain name that DNS servers store and return.

Some important DNS records:-

A record:- Contains the ipv4 address of a domain.

AAAA record:- Contains the ipv6 address of a domain.

NS record:- info about the authoritative dns servers
ex:- ns1.google.com

Mx record:- Tells you which mail servers handle email for a domain.

PTR record:- contains the domain name associated with a ip. So the opposite of A ^{or AAAA} records.
[IP address → domain]

CNAME record:- contains alias for a domain

[basically another name for the same thing]

Format:- ↓

[alias IN CNAME real-name]

TXT record :- contains text data associated with a domain.

ex:-

example.com IN TXT "Only Server X is allowed to send email for this domain"

↳ Mail servers read this text & help use it to help detect forged emails

(1)

HTTP PUT:- used to replace or update an existing resource

ex:- {
 "name": "John Smith"
 "email": "john@example.com"
}

↳ Replacing the data for a user with this new data

HTTP DELETE:- Used to remove a resource

ex:- DELETE /users/123
 ↓
 delete this user

HTTP is usually stateless meaning every request is independent

Basically server doesn't remember you

Cookies:- small piece of data stored by the browser

When you log in server sends:-

↳ Set-cookie: session-id=abc123

↳ Browser stores it

Now every future request to the website includes the cookie

ex:-

GET /profile

Cookie: session-id=abc123

This cookie contains the user data

Now you don't need to login with your credentials every single time.

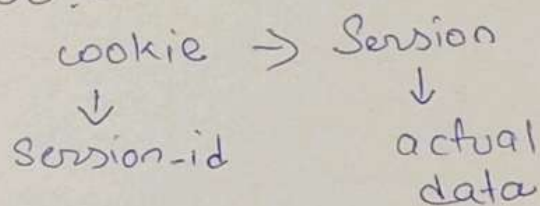
Session:- This is data stored by a server to remember information about a particular client

like username
password

cookies are created so later when the server receives it, it can check which session this user request belongs to.

Both work together to ^{promote} provide dependent requests

So:-



TLS (Transport Layer Security) :- Its a protocol that provides authentication, key exchange and encryption for network communication

TLS handshake :- This happens right after the establishment of TCP connection.

Flow of TLS handshake



First on both client & server a secret private key is generated and then a public keys derived from those private keys are generated.



First the client sends :-

- TLS version supported
- Encryption algorithm supported
- client's public key

→ [I want a secure connection. Here are the options I support]



Then after receiving the client info the Server sends :-

- TLS version chosen
- Encryption algorithm chosen
- Its ^{Server's} public key
- Certificate

→ [Lets use these settings]



Certificate contains info like:-

- Website name
- Public Key
- Certificate authority signature



Browser verifies the certificate → helps to know if its a valid server



As both sides have their public keys they use it to generate a secret key

On client's side → Its private key + Server's public key = Shared key

Same other side

Both end up with same shared key

To understand this process, simple example



lets say client chooses



Both sides send messages "I successfully generated the shared key"



Handshake complete

How this process is encrypted?

↓
we will use a very simple example

↓
lets assume

client's → secret colour: Red

Server's → secret colour: Blue

Shared colour: Yellow

already generated on both sides after the sharing of public colours

client sends a mixture → Yellow + Red

Server sends a mixture → Yellow + Blue

Hackers can't see the private colour because it's mixed with the shared colour

↓
Client computes (Yellow + Blue) + Red
receiving server's info

Server computes (Yellow + Red) + Blue
after receiving client's info

As you notice at the end both will end up with a same colour and the attacker will have no idea of either the secret or the final message.

We used colours to simplify understanding
In real world, we use mathematical operations
Instead of colours. Just know we use maths
to generate every thing discussed with colours

That colour mixing $\rightarrow$ mathematical operation
called a key exchange
algorithm (ECDHE)

Common ports

(1)

Port 21 - FTP (File transfer protocol)

↳ A service for uploading and downloading file

↳ Sometimes login without password

↳ May contain files, backups, credentials, source code.

Port 22 - SSH (Secure Shell)

↳ Remote command line access to a machine with encryption

↳ Common on Linux servers

↳ If you obtain valid credentials u can get a shell on the target

Port 23 - Telnet

↳ Works the same as SSH but not encrypted

↳ Username & pass may travel in plain text

↳ Often found on old devices and IoT equipment.

Port 25 - SMTP

- ↳ Email transfer service
- ↳ Sometimes leaks user info
- ↳ Can reveal valid usernames
- ↳ Useful for reconnaissance

Port 53 - DNS

- Can reveal subdomains and internal info
- Misconfigured servers may expose a lot of data

Port 80 - HTTP

- ↳ web traffic

Port 443 - HTTPS

- ↳ Encrypted HTTP

Port 445 - SMB (Server Message Block)

- ↳ Purpose is sharing files, folders, printers and other resources between computers on a network

↳ Windows relies on SMB for :-

- Shared folders
- Shared printers
- Accessing files on other computers

↳ Can

- Can reveal shares, usernames, computers & domain info

Port 3306 - MySQL

(2)

↳ Database service

- Databases may contain usernames, password hashes & sensitive info

Port 3389 - RDP (Remote Desktop Protocol)

↳ Remote graphical access to Windows

- If credentials are obtained you can log into the machine's desktop